
Modular Programming – Practical Work

Week 1

Recap exercises from 631-1

Notations: W2Q1 = Week 2, Questions of type 1

Question types:

- Type 1: theory, concepts *knowledge*
- Type 2: understanding typically based on code reading *understanding*
- Type 3: producing yourself code *know-how*

A moodle deposit space is open every week for you to upload your solutions. Solutions of the exercises are provided one week later.

Important points:

- 602_F and 602_D classes should upload solutions **before Wednesday 0h00 the following week**
- **2 or 3 practical work will be corrected** and will count for one third of the intermediate test note

Conventions about fonts:

- `Lucida Console` is used for Java reserved commands
- `Courier` is used for the code
- Arial is used for the text

W1Q1 – 1

1. What is the difference between an algorithm and a program?
2. Why does the expression $5+3*2$ gives 11 and not 16? How does a programming language handle this?

W1Q1 – 2

1. What is happening if you define a for loop like this:

```
for(short i=0;i<100000;i++)
```

2. I want to define a 2 dimensional int array as follows:

- 3 rows
- row 1 has 6 columns, row 2 has 5 columns and row 3 has 4 columns

Is it possible in Java?

W1Q2 – 1

What does the following code display? Underline the correct answer.

```
String myString ;  
myString = "jean-claude" ;  
myString.replace('j', 'J') ;  
System.out.println(myString) ;
```

- a) jean-claude
- b) Jean-claude

W1Q2 – 2

What does this program display? Give the answer without running this code. Read the code as if you were Eclipse's debugger.

```
public class X {  
  
    public static void main(String[] args) {  
        int array[] = {8,3,5,6,4,7} ;  
        mystery(array) ;  
        display(array) ;  
    }  
  
    public static void mystery(int array[])  
    {  
        int length=array.length;  
        boolean permut;  
  
        do  
        {  
            permut=false;  
            for(int i=0;i<length-1;i++)  
            {  
                if(array[i]>array[i+1])  
                {  
                    move(array,i,i+1);  
                    permut=true;  
                }  
            }  
        }  
        while(permut);  
    }  
}
```

```

public static void move(int array[], int pos1, int pos2)
{
    int temp = array[pos1] ;
    array[pos1] = array[pos2] ;
    array[pos2] = temp ;
}

public static void display(int [] array)
{
    for (int i = 0 ; i<array.length ; i++)
        System.out.print(array[i] + " ");
}
}

```

W1Q2 – 3

What does this method display if we have “hello” and 3 for input? Please give the answer without running this code or checking the results on the internet. Read the code as if you were Eclipse’s debugger.

```

public static String mystery(String input, int shiftValue)
{
    String result = "";

    char[] charArray = input.toCharArray();
    int[] outputArray = new int[charArray.length];

    for( int i=0; i<charArray.length; i++ )
    {
        if(charArray[i] == ' ')
        {
            outputArray[i] = charArray[i];
        }
        else
        {
            int shifted = (charArray[i] + shiftValue);
            outputArray[i] = shifted ;
        }
        result += Character.toString((char) outputArray[i]);
    }
    return result;
}

```

You can now start Eclipse. Q3 means questions of type 3 that involve programming with Eclipse.

W1Q3 – 1

Read and understand what the following code does and then implement the same result using while loops instead of for loops.

```
for (int i = 1; i < 3; i++ )
{
    for (int j = 3; j > 0; j-- )
    {
        for ( int k = 1; k <= 6; k=k+2 )
            System.out.print( '*' );

        System.out.println();
    }
    System.out.println();
}
```

W1Q3 – 2

Write an application that plays "guess my number" as follows: Your application chooses the number to be guessed by selecting randomly an integer in the range 1 to 1000. The application then displays the following text:

```
I have a number between 1 and 1000. Can you guess my number?
Please enter your first guess.
```

The user is entering an integer using the keyboard and the program displays either "Too High" or "Too Low" according to the value of the guess. When the user gets the correct answer, "Correct!" should be displayed.

W1Q3 – 3

Go back to the W1Q2-3 question and write the decrypt method that will permit you to have the first input back. In that case, it will permit you to have the word "hello" again.

W1Q3 – 4

*** deutscher Text folgt ***

Un restaurant est rempli de tables de différentes tailles. Lorsque des clients arrivent au restaurant, le nombre de personnes est demandé. S'il y a une table assez grande de libre, ils peuvent s'asseoir à cette table. Cette table est alors occupée.

Votre tâche consiste à écrire un programme qui gère les tables d'un restaurant. Pour ce faire vous allez réaliser les tâches suivantes :

- a. Créer une classe **Table**. La classe doit gérer le nombre de chaises d'une table, le statut (« occupé » ou « libre ») et le numéro de la table (numéro unique). Implémenter également un constructeur avec des paramètres, des getters et des setters ainsi qu'une méthode **toString()** qui retourne les informations sur la classe sous forme de string. Initialement, le statut est toujours « libre ».
- b. Implémenter un deuxième constructeur qui fixe le nombre des chaises à 4.
- c. Implémenter une classe **Restaurant** qui contient une fonction **main()** pour simuler un restaurant. Au début, demander le nombre de tables et le nombre de chaises pour chaque table. Gérer les tables dans un tableau.
- d. Créer dans la classe de test, un menu avec les options suivantes : « s'asseoir », « partir », « afficher », « fin ». Les options sont détaillées ci-après :
- e. Option « s'asseoir » : Attribuer une table libre aux clients qui viennent d'arriver (l'utilisateur indique toujours le nombre de personnes). Toujours attribuer la première table disponible pour le nombre de personnes requises. La table a alors le statut « occupée ».
- f. Option « partir » : Le numéro de la table est indiqué et le statut de la table passe à « libre ».
- g. Option « afficher » : Afficher les informations sur toutes les tables gérées.
- h. Si un client veut partir d'une table qui n'est pas disponible dans le restaurant, un message d'erreur doit être affiché.

In einem Restaurant stehen Tische mit jeweils einer unterschiedlichen Anzahl Stühle. Gäste werden beim Eingang nach der Anzahl der Personen gefragt, die an einem Tisch Platz nehmen möchten. Ist ein Tisch frei, können sie sich an den zugewiesenen Tisch setzen – dieser Tisch ist nun belegt. Schreiben Sie ein Programm, das die Tische im Restaurant verwaltet:

- a. Erstellen Sie eine Klasse **Table**. Verwalten Sie die Anzahl der Stühle am Tisch und den Status belegt bzw. nicht belegt sowie eine eindeutige Tischnummer. Implementieren Sie einen Konstruktor mit Parametern, Gettern und Settern sowie einer Methode **toString()**, mit der die Informationen über die Klasse als String zurückgegeben werden. Der Status ist zu Beginn immer „frei“.
- b. Implementieren Sie einen zweiten Konstruktor, der die Anzahl der Stühle auf 4 festlegt.
- c. Implementieren Sie eine Klasse **Restaurant** mit dem **main()**. Fragen Sie zu Beginn nach der Anzahl der Tische und für jeden Tisch nach der Anzahl der Stühle. Verwalten Sie die Tische in einem Array.
- d. Erstellen Sie ein Menü mit der Auswahl „setzen“, „aufstehen“, „anzeigen“ und „Ende“.
- e. Auswahl „setzen“: Weisen Sie nun ankommenden Gästen (der Anwender gibt jeweils die Anzahl der Personen ein) einen freien Tisch zu. Weisen Sie jeweils den ersten Tisch zu, der in Bezug auf die Anzahl der Personen möglich ist. Der Tisch hat nun den Status „belegt“.
- f. Auswahl „aufstehen“: Unter Angabe der Tischnummer wird der Status für diesen Tisch auf „frei“ gesetzt.
- g. Auswahl „anzeigen“: Zeigen Sie die Informationen über alle verwalteten Tische an.
- h. Sollte der Anwender von einem Tisch aufstehen wollen, der nicht im Restaurant verfügbar ist, geben Sie eine Fehlermeldung aus.

Exemple d'exécution / Beispiel :

How many tables in your Restaurant? 3
How many persons can seat around the table 1 ? 6
How many persons can seat around the table 2 ? 4
How many persons can seat around the table 3 ? 8
Choose:
1) Take a seat
2) Go away
3) Show
4) End
1
How many persons are you ?
7
You can seat around the table number 3
Choose:
1) Take a seat
2) Go away
3) Show
4) End
1
How many persons are you ?
4
You can seat around the table number 1
Choose:
1) Take a seat
2) Go away
3) Show
4) End
1
How many persons are you ?
7
All tables are occupied, wait for a moment
Choose:
1) Take a seat
2) Go away
3) Show
4) End
3
The table 1 with 6 seat(s) ist occupied
The table 2 with 4 seat(s) ist free
The table 3 with 8 seat(s) ist occupied
Choose:
1) Take a seat
2) Go away
3) Show
4) End
4